

Encoders

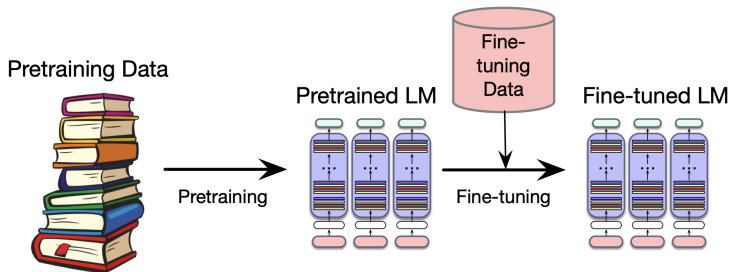
What Are Foundation Models?

- ▶ Large neural networks trained on massive corpora using self-supervised objectives (learning from tons of text without explicit labels).
- ▶ Learn general-purpose representations of language (capturing patterns in how words and sentences are structured).
- ▶ Serve as reusable backbones for many downstream NLP tasks.
- ▶ Typically trained once (expensive) and adapted many times (cheap).
- ▶ Most modern foundation models are based on the Transformer architecture
 - ▶ originally introduced for sequence modeling
 - ▶ central to contextual word representations

Word Representations

- ▶ Machine learning models require numerical input.
- ▶ Raw text must be converted into vectors.
- ▶ Naïve representations (e.g., one-hot encoding):
 - ▶ Do not encode similarity.
 - ▶ Do not generalize.
 - ▶ Ignore context.
- ▶ Word2Vec-style embeddings (2013):
 - ▶ Dense, low-dimensional vectors instead of sparse one-hot vectors.
 - ▶ Words that appear in similar contexts get similar representations.
 - ▶ One fixed vector per word (static embeddings).
- ▶ Contextual embeddings (2017):
 - ▶ The representation of a word depends on its surrounding context.
 - ▶ “bank” in “river bank” differs from “bank” in “investment bank”.
 - ▶ Representations transfer across tasks.

How Foundation Models are made and used



H. Ji, Advanced Topics in Natural Language Processing, Univ. Illinois

What is an encoder?

- ▶ An encoder is a parametrized function whose behavior depends on learned parameters θ :

$$f_{\theta} : \mathbb{R}^{n \times d} \rightarrow \mathbb{R}^{n \times d}$$

- ▶ In practice, the full pipeline is:

text $\rightarrow$ tokens $\rightarrow$ embeddings $\rightarrow$ contextual representations

- ▶ The encoder operates on token embeddings and produces contextualized vectors (non-contextual vectors in, context-dependent vectors out).
- ▶ Output:
 - ▶ One contextual vector per token.
 - ▶ Optionally, a global sequence representation (a single vector intended to summarize the whole sequence, often used for classification tasks).

How are encoders built?

- ▶ Modern encoders are built from stacked Transformer encoder layers.
- ▶ Key mechanism: **self-attention**
 - ▶ Each token representation is updated by attending to all other tokens.
 - ▶ The model asks: “Which words in this sentence matter for interpreting this token?”
 - ▶ It assigns attention weights to all tokens and aggregates information accordingly.
 - ▶ Self-attention is permutation-invariant; positional embeddings inject information about token order.

BERT¹

Bidirectional Encoder
Representation from
Transformers



Breakthrough paper that
demonstrated the power of
transformers

- ▶ Encoder-only architecture
- ▶ Pre-train deep bidirectional representations by jointly conditioning on both left and right context in all layers
- ▶ Pre-trained BERT representations can be fine-tuned with just one additional output layer to create models for a wide range of tasks without substantial task-specific architecture modifications

¹J. Devlin et al., BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, ArXiv pre-print, 2018

Attention Review

The attention mechanism simulates a database query.

- ▶ A database can be viewed as a collection of *key:value* pairs.
- ▶ Given a query q , we want to retrieve the value v associated with the most relevant key k .
- ▶ In self-attention, every token produces:
 - ▶ a **query** vector,
 - ▶ a **key** vector,
 - ▶ a **value** vector.

Attention Review

- ▶ Similarity between queries and keys:

$$QK^T$$

(dot products between query vectors and key vectors)

- ▶ Scaled and normalized attention weights:

$$\text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

(d_k = dimensionality of key vectors)

- ▶ Final output:

$$\text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

a weighted average of the value vectors.

Attention: more intuition

- ▶ We project each embedding entering the attention layer into three learned vector spaces:
 - ▶ **Query (Q)**: what kind of information this token is looking for in other tokens.
 - ▶ **Key (K)**: how relevant this token is to other tokens' Queries.
 - ▶ **Value (V)**: what this token contributes if it is attended to.
- ▶ Attention weights are computed from Query–Key similarities, and these weights are used to aggregate the Values.
 - ▶ For each token, we compute similarity scores between its Query and all Keys.
 - ▶ After normalization (via softmax), these scores determine how much attention the token pays to every other token.
 - ▶ We use these normalized scores to compute a weighted average of the Value vectors.
 - ▶ The result is a new representation that emphasizes the most contextually relevant information.

Attention - Simple example

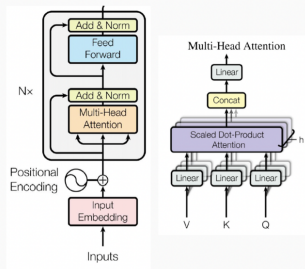
Dan didn't cross the road because it was too wide.

- ▶ We focus on the token *it*.
- ▶ Its **Query** encodes features associated with something that can be *wide*.
- ▶ The model compares this Query to the **Keys** of all other tokens:
 - ▶ *road* → high similarity (its representation aligns with *wide*)
 - ▶ *Dan* → low similarity (its representation aligns less with *wide*)
- ▶ After softmax normalization, *road* receives a high attention weight.
- ▶ The new representation of *it* becomes a weighted combination of the **Values**, dominated by the Value of *road*.

Result: The model resolves that *it* refers to *road* through attention.

BERT architecture

- Multi-headed self attention
 - Models context
- Feed-forward layers
 - Computes non-linear hierarchical features
- Layer norm and residuals
 - Makes training deep networks healthy
- Positional embeddings
 - Allows model to learn relative positioning



Source: J. Devlin, Google

Multi-Head Attention (MHA)

- ▶ Input $X \in \mathbb{R}^{n \times d}$ is linearly projected:
 $Q = XW_Q$, $K = XW_K$, $V = XW_V$.
- ▶ Split Q, K, V into A heads of size $d_h = d/A$.
- ▶ Apply scaled dot-product attention independently in each head.
- ▶ Concatenate the A head outputs.
- ▶ Apply final projection W_O to return to dimension d .

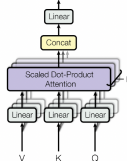
- ▶ Different heads learn different relationships (syntax, long-range dependencies, coreference).
- ▶ Projection cost:

$$A \times d \times \frac{d}{A} = d^2$$

Scaled Dot-Product Attention



Multi-Head Attention



BERT Base: $H = 768$, $A = 12$,
 $\frac{H}{A} = 64$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Other components

Position-wise FFN

- ▶ Applied independently to each token.
- ▶ Two linear layers with nonlinearity:

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

- ▶ Expands and contracts dimension:

$$d \rightarrow 4d \rightarrow d$$

BERT Base: $768 \rightarrow 3072 \rightarrow 768$

Residual connections

$$y = x + \text{Sublayer}(x)$$

- ▶ Preserve input signal

Layer normalization

$$\text{LayerNorm}(h) = \gamma \frac{h - \mu}{\sigma} + \beta$$

- ▶ Normalizes across feature dimension
- ▶ Stabilizes training when there are many layers

BERT's Bidirectional Language Modeling

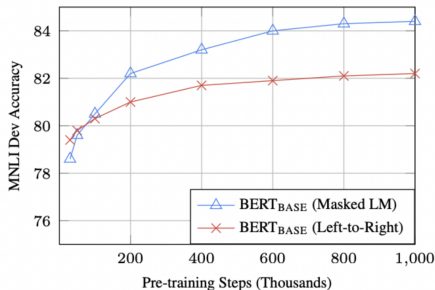
Key idea: Train a bidirectional language model by masking a fraction of the input tokens and asking the model to predict them.

store gallon
↑ ↑
the man went to the [MASK] to buy a [MASK] of milk

- ▶ If too few tokens are masked, training is computationally inefficient.
- ▶ If too many tokens are masked, the model lacks sufficient context.
- ▶ Empirically, masking 15% of tokens provides a good trade-off.

Impact of bidirectional context

- MLM converges slower
- But it outperforms left-to-right LM



Adapting pretrained BERT for downstream tasks

We adapt a pretrained BERT model for a specific downstream task (e.g., classification) by adding and training an additional task-specific layer on top.

Step 1: Input preparation

- ▶ The input sentence (or full text) is tokenized and fed into BERT.
- ▶ A special token [CLS] is added at the beginning of the sequence.
- ▶ The [CLS] token serves as a summary representation of the entire input.

Adapting pretrained BERT for downstream tasks

Step 2: Pass through BERT

- ▶ The tokenized input is passed through BERT's transformer encoder layers.
- ▶ Each token (including [CLS]) is transformed into a contextual embedding.
- ▶ These embeddings capture the meaning of each token in context.
- ▶ The final-layer embedding of [CLS] is used as a representation of the entire sentence for classification.

Adapting pretrained BERT for downstream tasks

Step 3: Add a classification head

- ▶ Add a thin layer of neurons on top of BERT.
- ▶ Typically, this is a fully connected (feedforward) layer applied to the [CLS] embedding.
- ▶ The [CLS] token contains aggregated information about the entire input text.
- ▶ The classification layer maps this representation to task-specific outputs (e.g., *Positive* / *Negative*).

Example: Text Classification

Example: Classifying news articles into *Sport*, *Politics*, and *Fashion*

- ▶ The classification head consists of:
 - ▶ A fully connected layer with 3 neurons (one per category).
 - ▶ A softmax activation function to convert logits into class probabilities.

Mathematically:

$$\hat{y} = \text{softmax}(Wh_{[\text{CLS}]} + b),$$

where $W \in \mathbb{R}^{3 \times d}$ and $h_{[\text{CLS}]} \in \mathbb{R}^d$.

Linear algebraic view

In linear algebraic terms, the 3 neurons are the 3 components of $\mathbf{z}$, which are then fed into a softmax

$$\mathbf{z} = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1d} \\ w_{21} & w_{22} & \dots & w_{2d} \\ w_{31} & w_{32} & \dots & w_{3d} \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_d \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

[CLS]
embedding
from BERT

Each component is a raw score called **logit**

$$z_1 = w_{11}h_1 + w_{12}h_2 + \dots + w_{1d}h_d + b_1$$

$$z_2 = w_{21}h_1 + w_{22}h_2 + \dots + w_{2d}h_d + b_2$$

$$z_3 = w_{31}h_1 + w_{32}h_2 + \dots + w_{3d}h_d + b_3$$

Inference after fine-tuning

After fine-tuning, the model is ready for inference.

- ▶ Provide a new input text.
- ▶ BERT encodes the sequence.
- ▶ The final-layer embedding of the [CLS] token is passed to the classification head.
- ▶ The classification head produces a probability distribution:

$$\hat{y} = \text{softmax}(Wh_{[\text{CLS}]} + b)$$

- ▶ The predicted class is:

$$\arg \max_k \hat{y}_k$$

Next Sentence Prediction (NSP)

BERT is pre-trained with two tasks:

- ▶ Masked Language Modeling (MLM)
- ▶ **Next Sentence Prediction (NSP)**

NSP training objective

- ▶ Input:

[CLS] *A* [SEP] *B* [SEP]

- ▶ The model predicts whether sentence *B* is the actual next sentence in the corpus after sentence *A*
- ▶ The NSP classifier is applied to the vector

$$C = h_{[\text{CLS}]}$$

- ▶ The [CLS] representation learns to capture information about the entire sequence.

Questionable design choices

Later work² questioned two BERT design choices:

- ▶ Next Sentence Prediction (NSP)
- ▶ Using the [CLS] token as the primary sentence representation

Key findings:

- ▶ NSP contributes little to downstream performance.
- ▶ Sentence representations can be obtained more effectively by pooling token embeddings.

²For example, Liu et al. (2019), RoBERTa: A Robustly Optimized BERT Pretraining Approach

Common alternatives to [CLS]

- ▶ **Mean pooling**

$$s = \frac{1}{n} \sum_{i=1}^n h_i$$

- ▶ **Max pooling**

$$s_j = \max_i(h_{i,j})$$

- ▶ Mean pooling captures the average contextual meaning of the sentence, while max pooling captures the strongest signal in each dimension of the representation.
- ▶ These approaches treat the sentence embedding as a **summary of all token representations** rather than relying on a special token.

BERT for Sentences: Efficiency Problem

Two sentences must be encoded **together**:

$$[\text{CLS}] s_1 [\text{SEP}] s_2 [\text{SEP}]$$

The model processes the pair jointly and predicts whether the sentences are related.

To compare a collection of N sentences, we must evaluate **all sentence pairs**.

$$\text{Number of pairs} = \frac{N(N-1)}{2}$$

which grows approximately as

$$O(N^2)$$

Each comparison requires a full BERT forward pass. This happens because BERT processes the two sentences together inside the transformer.

This becomes impractical for large-scale semantic search.

BERT for Sentences: Representation Problem

BERT was not trained to produce good sentence embeddings.

- ▶ Pre-training objectives (MLM, NSP) do not optimize semantic similarity.
- ▶ Sentence representations are not explicitly learned.
- ▶ The embedding space becomes **anisotropic**.

Anisotropy

Most embeddings lie in a narrow cone of the vector space.

Consequences:

- ▶ cosine similarity between unrelated sentences is often relatively high
- ▶ semantic similarity becomes hard to distinguish

Sentence-BERT addresses this by **training embeddings directly for similarity**.

Sentence-BERT³

Sentence-BERT adapts BERT to produce **sentence embeddings**.

Each sentence is mapped to a vector:

$$s \rightarrow f(s) \in \mathbb{R}^d$$

Similarity between sentences is computed with

$$\text{cosine}(f(s_1), f(s_2))$$

Advantages

- ▶ sentences can be **encoded independently**
- ▶ embeddings can be **precomputed**
- ▶ similarity reduces to a fast **vector comparison**
- ▶ enables large-scale **semantic search**

³<https://aclanthology.org/D19-1410.pdf>

Sentence Embeddings as a Function

Sentence-BERT learns a function

$$f(s) : \text{sentence} \rightarrow \mathbb{R}^d$$

that maps *any* sentence to a vector.

Examples:

$$\text{"A dog is running"} \rightarrow f(s_1)$$

$$\text{"A puppy is playing"} \rightarrow f(s_2)$$

During training the model learns a geometry where

- ▶ semantically similar sentences $\Rightarrow$ nearby vectors
- ▶ unrelated sentences $\Rightarrow$ distant vectors

Precomputing Sentence Embeddings

For a corpus of N sentences we compute embeddings once:

$$s_i \rightarrow f(s_i) = v_i$$

and store the vectors $v_1, \dots, v_N$.

When a new query q arrives:

$$q \rightarrow f(q) = v_q$$

Similarity is computed using

$$\text{cosine}(v_q, v_i)$$

Only the query requires a new BERT forward pass.

Siamese Encoder Architecture

Sentence-BERT uses a **bi-encoder (siamese) architecture**.

Two identical encoders with **shared weights**:

$$s_1 \rightarrow \text{BERT} \rightarrow f(s_1)$$

$$s_2 \rightarrow \text{BERT} \rightarrow f(s_2)$$

Training shapes the embedding space so that

- ▶ semantically similar sentences have embeddings that are **close**
- ▶ unrelated sentences have embeddings that are **far apart**

Training Sentence Embeddings

Training forces embeddings to respect semantic similarity.

Example triplet:

- ▶ Anchor: "A man is eating."
- ▶ Positive: "Someone is eating food."
- ▶ Negative: "A car is parked."

Goal:

$$d(a, p) < d(a, n)$$

Triplet loss:

$$L = \max(0, d(a, p) - d(a, n) + m)$$

where m is a margin.

Cross-Encoder vs Bi-Encoder

Cross-Encoder (BERT)

$[\text{CLS}] s_1 [\text{SEP}] s_2 [\text{SEP}]$

- ▶ both sentences processed together
- ▶ deep token-level interaction
- ▶ must run the model for every pair

Bi-Encoder / Siamese Encoder (SBERT)

$s_1 \rightarrow f(s_1)$

$s_2 \rightarrow f(s_2)$

- ▶ sentences encoded independently
- ▶ embeddings compared using cosine similarity
- ▶ embeddings can be precomputed

Fast Semantic Search

With SBERT:

- ▶ encode each sentence once

$$s_i \rightarrow f(s_i)$$

Then compute cosine similarity between vectors.

SBERT still computes contextual embeddings for subword tokens, but then pools them into a single vector representing the whole sentence.

Intuitive takeaways

- ▶ BERT is kind of like a very knowledgeable reader that has gained some meaningful level of understanding of human language through sheer exposure.
- ▶ Fine-tuning teaches our reader to specialize in a specific task, like identifying sentiment or detecting spam, by adding a simple decision-making layer on top.

From BERT to Large Language Models (LLMs)

- ▶ **BERT (2018):**

- ▶ Used primarily for fine-tuning on specific tasks.

- ▶ **Scaling up:**

- ▶ More data, larger models, and more parameters lead to better generalization.
- ▶ Transformer-based models grew in size and capability.

- ▶ **Large Language Models (LLMs):**

- ▶ GPT-3 (2020) introduced autoregressive training at scale.
- ▶ Capable of few-shot as well as even zero-shot learning.

BERT rediscovers the classical NLP pipeline (Tenney et al., 2019)

Where is linguistic information stored inside BERT?

Edge Probing

- ▶ Freeze pretrained BERT.
- ▶ Train lightweight classifiers on top of each layer.
- ▶ Tasks span the classical NLP pipeline:
 - ▶ POS
 - ▶ Constituency
 - ▶ Dependencies
 - ▶ NER
 - ▶ SRL
 - ▶ Coreference
- ▶ Measure which layers contain extractable information.

Goal: Detect *where* different linguistic abstractions emerge.

Measuring Layer Contributions

Layers organize information in pipeline order:

POS $\rightarrow$ Syntax $\rightarrow$ SRL $\rightarrow$ Coreference

Lower layers: local syntax.

Higher layers: semantics and discourse.

- ▶ BERT internally mirrors the traditional NLP pipeline.
- ▶ The ordering holds in aggregate, not rigidly.
- ▶ On individual examples:
 - ▶ Higher-level decisions revise lower-level ones.
 - ▶ Ambiguities are resolved dynamically.
- ▶ Semantic information is more distributed across layers.
- ▶ Syntactic information is more localized.

Reflections

Observations The state of the art is built primarily on Transformer architectures (2017) trained via backpropagation (1986).

- ▶ Architecturally, most frontier systems remain Transformer-based, with innovation concentrated in scale, training strategy, alignment methods, and system-level engineering.
- ▶ No new foundational optimization principles (e.g., a backpropagation-level advance) have emerged.

Concerns

- ▶ Scaling has physical and economic limits.
- ▶ Diminishing returns.
- ▶ Increasing amounts of synthetic data will reduce training signal quality.
- ▶ If the foundation remains unchanged, the trajectory will face structural ceilings.

Reflections

Counterarguments

- ▶ It may be tempting to dismiss them as an engineering hack that actually worked, but Transformers have brought a paradigm shift.
- ▶ Scaling has unlocked unexpected emergent capabilities.
- ▶ Synthetic data concerns may be exaggerated.
- ▶ We may be in a refinement phase preceding another conceptual breakthrough.